Bug Report Classification with a Linear SVM and Enriched Text Features

Intelligent Software Engineering Coursework – Tool Building Project

Kanishk Suri (Student ID: 2970058)

1 Introduction

Most bug trackers receive a steady volume of free-form text, and the categorisation of that text matters because nearly every downstream task assumes it has already been done correctly. Triage, assignee selection, duplicate detection and fix-time estimation all build on top of the category label [1]. Volume is part of the difficulty: Eclipse, between v3.0 and v3.1, accumulated 13,016 reports at an average of 37 per day with spikes above 220 [1], while Mozilla was processing around 300 per day as far back as 2005 [2]. Manual triage at those rates is not feasible, and an incorrect label is expensive because the cost is paid by every downstream step.

This study isolates the categorisation step itself. Two corpora are used. The first is a synthetic dataset of 3,000 reports built from a small set of sentence templates; the class boundaries are clean by construction, so the corpus serves as a measurement target for what the feature set can do without the confounding effect of noise. The second corpus is the four computer-related categories of 20 Newsgroups, comprising 3,918 Usenet posts, used as a noisier stand-in for technical-report text. The baseline classifier is Multinomial Naive Bayes over TF-IDF, following Limsettho et al. [3]. Against that baseline the report places a Linear SVM trained on a feature union of TF-IDF, word bigrams, character n-grams and eight hand-coded text statistics. The regularisation parameter of the SVM is selected on a held-out validation split prior to the main evaluation, so model selection and final evaluation never share data. Comparison itself relies on stratified 10-fold cross-validation, a separate 80/20 hold-out, and a two-tailed Wilcoxon signed-rank test [9] at $\alpha=0.05$.

The work is organised around three research questions. **RQ1:** on a controlled corpus with clean class boundaries, does a tuned LinearSVC over TF-IDF extended with word bigrams, character n-grams and text statistics outperform the NB plus TF-IDF baseline? **RQ2:** on a noisier corpus (20 Newsgroups), is the same gap still observable, and is it meaningful both in p-value and in effect size? **RQ3:** of the three feature groups added on top of unigram TF-IDF, which one accounts for most of the signal, and by how much? Section 7 revisits each question and provides a direct answer.

2 Related Work

Antoniol et al. (2008) [4] combined stemming and indexing on top of TF-IDF, and evaluated Decision Tree, Naive Bayes and Logistic Regression classifiers on the resulting representation. Anvik et al. [1] reached around 64% accuracy when applying NB to the related Eclipse triage problem. A practical observation about bug-report text is that raw term frequency sometimes outperforms IDF-weighted variants, since tokens such as ‘crash’ or ‘failure’ remain class-informative irrespective of their global corpus frequency [2]. NB with TF-IDF still serves as the default baseline because it is fast and handles sparse, high-dimensional input well. Its principal weakness is the conditional-independence assumption, which prevents multi-token phrases such as ‘kernel panic’ from contributing their full discriminative weight, even where the phrase is a near-deterministic indicator of an OS-related defect.

Joachims [5] reported that SVMs typically outperform NB on text classification, and Limsettho et al. [3] observed the same pattern on bug reports from several open-source projects, which motivates the SVM adopted in this report. DRONE [6] takes a different direction by augmenting the report text with structured metadata, namely reporter history, component, version and severity. Such metadata is informative when it is available, although it presumes a richer dataset than most projects expose. The canonical reference for sub-word features is Cavnar and Trenkle [7]. Higher up the modelling hierarchy, MHNurf [8] is a hierarchical attention model with parallel word and character streams, which depends on GPU compute. Recent approaches that fine-tune pre-trained transformers [12] also require GPU resources. The tool presented here is intentionally CPU-only, so the design remains within the sparse TF-IDF family. Table 1 summarises the alternatives that were considered.

Family	Examples	Pros	Cons
NB + TF-IDF	[1, 3, 4]	Fast; handles sparse input	Independence assumption; ignores word order
DT / LR + TF-IDF	[4]	Interpretable; LR gives probabilities	Lower accuracy than SVM on text [5]
Linear SVM + TF-IDF	[3, 5]	Strong on high-dim text; margin objective	No probabilities; needs feature scaling
Linear SVM + word/char n-grams	this report, [7]	Catches sub-word patterns; tolerant of typos	Larger feature matrix; more memory
Hierarchical attention + GloVe	[8]	Captures semantics; multifaceted features	Pre-trained vectors; more compute
Pre-trained LM (BERT-style)	Devlin et al. [12]	Best raw accuracy on noisy text	GPU and fine-tuning required

Table 1. Approaches to bug-report classification considered here.

3 Solution

3.1 Feature Engineering

Three feature groups are concatenated using scikit-learn’s FeatureUnion. The first is a word-level TF-IDF representation parameterised with ngram_range=(1,2), sublinear_tf=True, max_features=50,000 and min_df=2. Bigrams matter here because two-word collocations like ‘kernel panic’ (OS) or ‘null pointer’ (platform) carry considerably more discriminative weight than either token taken on its own, and sublinear TF scaling damps the pull of very frequent tokens that would otherwise dominate. The second is a character n-gram TF-IDF representation (analyzer=char_wb,

ngram_range=(2,4), max_features=30,000), which captures sub-word fragments such as BIOS, Xorg, ACPI and a number of driver suffixes that word-level tokenisation tends to split. Character features additionally tolerate typos and surface variation in noisier reports comparatively well [7]. The min_df threshold for this block was settled at 3 after an initial run with min_df=1 produced a vocabulary in excess of 80k columns and tripled the fit time. The third group is a fixed block of eight hand-coded text statistics per report: character length, word count, newline count, count of uppercase characters, count of digits, and the occurrence counts of the substrings ‘error’, ‘bug’ and ‘fix’. These are intended to surface structural cues, for instance the presence of long stack traces. The concatenated feature matrix is passed through a MaxAbsScaler prior to the SVM so that no single block dominates the loss simply because of differences in raw magnitude across its values.

3.2 Classifier and Hyperparameter Search

The classifier itself is scikit-learn’s LinearSVC, with one-against-rest multi-class handling, tol=1e-3 and max_iter=50,000. The choice of a linear kernel over an RBF kernel rests on two considerations. First, the feature space after the FeatureUnion is already high-dimensional and sparse, and in that regime a linear decision boundary is usually competitive with non-linear alternatives [5]. Second, the linear formulation sidesteps the extra runtime incurred by a kernel SVM. Relative to NB, an additional advantage is that the SVM margin objective does not rest on a feature-independence assumption, which means that the correlated bigram and character-n-gram features can actually be used rather than penalised. Of the available hyperparameters, only the regularisation strength C has a non-trivial effect on text-classification performance in this setting. A five-point grid, $C \in \{0.1, 0.5, 1.0, 2.0, 5.0\}$, was evaluated on a stratified 75/25 internal validation split at seed 42, with the winning C chosen on Macro-F1. The selected value is then frozen for the rest of Section 5. Table 2 shows that the selected C is 0.1 on both datasets. Keeping the HPO split disjoint from the splits used in the main evaluation is deliberate; it removes the possibility of accidentally tuning on the test data. The full pipeline is therefore FeatureUnion(word_tfidf, char_tfidf, text_stats) → MaxAbsScaler → LinearSVC(C=C*).

Dataset	C=0.1	C=0.5	C=1.0	C=2.0	C=5.0	C* selected
Synthetic	1.0000	1.0000	1.0000	1.0000	1.0000	0.1
20 Newsgroups	0.8103	0.8045	0.8045	0.8055	0.8065	0.1

Table 2. Macro-F1 on the 25% internal validation split for each candidate C. Selected C is the column with the highest validation F1 (ties broken by the smaller C, which is the more regularised choice).

4 Setup

Datasets. Two corpora are used. The first is a synthetic set of 3,000 reports drawn from a small template library; six sentence templates per class, together with shared lists of components, error names and version strings. The four classes (hardware_bug, os_bug, platform_bug and ui_bug) are balanced at exactly 750 reports each, which makes the corpus useful for measuring feature behaviour without confounding noise. The second corpus is the four computer-related categories of 20 Newsgroups, retrieved using sklearn.datasets.fetch_20newsgroups, namely comp.sys.ibm.pc.hardware, comp.os.ms-windows.misc, comp.sys.mac.hardware and comp.windows.x. This produces 3,918 Usenet posts mapped onto the same four labels. Headers, footers and quoted replies are stripped during the fetch. Using these four Newsgroup categories as proxies for tracker sub-areas avoids the licensing and labelling difficulties that would arise from scraping a live bug tracker. The random seed is fixed at 42 across the whole pipeline, so the run is fully deterministic.

Baseline. Multinomial Naive Bayes on unigram TF-IDF features, with max_features=50,000, English stop-word removal, min_df=2 and α=1.0, following Limsettho et al. [3]. Both classifiers are evaluated under identical conditions on the two corpora.

Metrics. Three metrics are reported on each fold. Accuracy is included as the standard headline measure that most text-classification papers report. Macro-F1 weights each of the four classes equally, which prevents a weak class from being hidden behind a high overall accuracy. The Matthews Correlation Coefficient (MCC) [10] is also reported because it uses every cell of the confusion matrix, and so gives a more conservative summary than F1 in multi-class problems where one class quietly drags down the rest.

Protocol. The main evaluation is stratified 10-fold cross-validation at seed 42. Stratification holds the class proportions roughly constant across folds, which is more important on 20 Newsgroups, where the four categories are not perfectly balanced. A separate stratified 80/20 hold-out (Section 5.3) is also run, and provides the data on which the timing measurements are taken. The hyperparameter grid in Section 3.2 uses its own 75/25 internal split, executed before any of the 10-fold runs, so the model selection process never observes data used for evaluation. Statistical comparison uses the two-tailed Wilcoxon signed-rank test [9] at α=0.05, applied to the 10 paired per-fold scores. Wilcoxon was selected because it is non-parametric, handles paired samples, and behaves reasonably at small n [11].

Environment. Apple M-series laptop CPU, 16 GB RAM, Python 3.13, scikit-learn 1.6, scipy 1.15. The full pipeline (data preparation, HPO, both classifiers, 10-fold CV, Wilcoxon, 80/20 hold-out, synthetic ablation and all figures) runs in approximately four minutes. No GPU is used at any stage.

5 Experiments

5.1 Cross-Validation Results

Table 3 reports the mean and standard deviation of each metric across the 10 stratified folds on both datasets, using the tuned C* from Section 3.2.

Dataset	Approach	Accuracy	Macro-F1	MCC
Synthetic	Baseline (NB + TF-IDF)	0.9793 +/- 0.0083	0.9793 +/- 0.0084	0.9726 +/- 0.0110

Dataset	Approach	Accuracy	Macro-F1	MCC
Synthetic	Solution (SVC + enriched, C*=0.1)	1.0000 +/- 0.0000	1.0000 +/- 0.0000	1.0000 +/- 0.0000
20 News.	Baseline (NB + TF-IDF)	0.8068 +/- 0.0110	0.8058 +/- 0.0111	0.7438 +/- 0.0145
20 News.	Solution (SVC + enriched, C*=0.1)	0.8101 +/- 0.0177	0.8099 +/- 0.0174	0.7474 +/- 0.0236

Table 3. 10-fold cross-validation results on both datasets after hyperparameter tuning.

On the synthetic corpus, the proposed solution beats the baseline on every fold, with margins of roughly +0.02 in Accuracy and Macro-F1 and +0.027 in MCC. The 97.9% baseline figure is already close to the achievable ceiling for this corpus, because the four synthetic classes draw on vocabulary that is distinct enough that a unigram bag-of-words is nearly enough on its own. On 20 Newsgroups both models settle around 81% Accuracy, which is consistent with the heterogeneous vocabulary, the off-topic posts that survive the cleanup and the overlap of technical jargon across the four categories. After tuning, the solution is slightly ahead in the mean values (+0.0033 Accuracy, +0.0041 Macro-F1, +0.0036 MCC), but those differences sit well inside the fold-to-fold standard deviation.

5.2 Statistical Significance

Dataset	Metric	W	p-value	Effect size |r|	Significant?
Synthetic	Accuracy	0.00	0.0020	1.000 (large)	Yes
Synthetic	Macro-F1	0.00	0.0020	1.000 (large)	Yes
Synthetic	MCC	0.00	0.0020	1.000 (large)	Yes
20 News.	Accuracy	23.00	0.6953	0.164 (small)	No
20 News.	Macro-F1	23.00	0.6953	0.164 (small)	No
20 News.	MCC	23.00	0.6953	0.164 (small)	No

Table 4. Wilcoxon signed-rank test (two-tailed, α=0.05) and rank-biserial effect size |r| computed from the W statistic and n=10 paired folds. Effect-size thresholds follow Cohen-style conventions: |r|≥0.5 large, 0.3–0.5 medium, 0.1–0.3 small [13].

On the synthetic corpus the test hits its lower bound on all three metrics. With 10 paired folds, W=0 and p=0.0020 are the smallest values that Wilcoxon can return, and the rank-biserial |r|=1.000 is the largest effect size attainable for this design, because every paired fold goes the same way. The synthetic comparison is therefore as significant, and as large in effect, as the chosen test allows. On 20 Newsgroups the picture inverts. The per-fold scores swap places repeatedly (W=23, p=0.6953), and the null hypothesis cannot be rejected at any conventional α. This is treated here as a genuine negative result rather than something to argue around with post-hoc analyses [11, 13]. The effect size, however, is small but non-zero (|r|=0.164): the solution wins on six of the ten folds, no folds tie, and the held-out 80/20 evaluation in Section 5.3 has the solution ahead on every metric. Taken together, the more defensible reading is that the solution is directionally better but underpowered at n=10 folds, not equivalent to the baseline. Two factors plausibly account for the smaller effect on real text. Real Usenet posts already contain a much wider surface vocabulary than the synthetic templates do, so most of what character n-grams add on synthetic data is already covered by the unigram bag on real data. The other factor is length: 20 Newsgroups posts run considerably longer than typical bug reports and include quoted replies and signatures that this pipeline makes no attempt to filter, which feeds noise into both classifiers in roughly equal measure.

5.3 Held-Out Test Set

Dataset	Model	Accuracy	Macro-F1	MCC
Synthetic	Baseline	0.9767	0.9766	0.9690
Synthetic	Solution	1.0000	1.0000	1.0000
20 News.	Baseline	0.8061	0.8054	0.7429
20 News.	Solution	0.8074	0.8077	0.7435

Table 5. Held-out 80/20 evaluation on both datasets after hyperparameter tuning.

The 80/20 hold-out broadly matches the cross-validation. On the synthetic corpus the baseline reaches 0.977 / 0.977 / 0.969 and the solution reaches 1.000 across all three metrics. On 20 Newsgroups the tuned solution finishes ahead on every metric (+0.0013 Accuracy, +0.0023 Macro-F1, +0.0006 MCC). The improvements are small, but they point in the same direction as the CV means. Full classification reports are printed by src/main.py at runtime, and the per-cell confusion matrices are written to results/cm_synthetic_*.png and results/cm_20news_*.png. The per-class 20 Newsgroups numbers are more informative than the headline aggregates. F1 on ui_bug rises from 0.84 to 0.93, F1 on hardware_bug nudges from 0.74 to 0.75, and F1 on platform_bug drops from 0.84 to 0.79. The redistribution of errors is consistent with the feature design: character n-grams are most informative when a class carries vocabulary of its own (‘Xorg’, ‘GTK’) and they help less when two classes draw on shared vocabulary, which is what happens in the Mac/PC hardware overlap.

5.4 Feature Contribution Check

Running the ablation on the full 80/20 synthetic split is uninformative, because every variant scores 1.000 there. The training set is therefore reduced to 25 reports per class, giving 100 in total, with the remaining 2,900 reports used for testing, and the same five configurations are then re-evaluated. Table 6 collects the resulting numbers. Only the feature pipeline is varied across rows; C is held fixed at the C* selected in Section 3.2 for every variant.

Configuration	Accuracy	Macro-F1	MCC
Full solution (word + char + stats)	0.9566	0.9567	0.9421
No character n-grams	0.9362	0.9359	0.9152
No text statistics	0.9555	0.9557	0.9408

Configuration	Accuracy	Macro-F1	MCC
Word unigrams only (drop bigrams)	0.9548	0.9550	0.9399
Word unigrams only, no char, no stats	0.9400	0.9399	0.9202

Table 6. Feature contribution check on the synthetic corpus, train=100 / test=2,900.

In this low-resource setting, character n-grams account for most of the performance gap. Removing the character block alone costs 0.020 Accuracy and 0.027 MCC. Bigrams look weak in the ablation, with a drop of only about 0.002 when they are removed, but the most natural explanation is sample size: 100 training reports do not give the model enough data to estimate bigram weights reliably. At full data scale, the bigrams do pay off, since the full pipeline in Section 5.1 reaches 1.000 while the unigram-only NB baseline sits at 0.9793. The text-statistics block contributes only marginally here (+0.001 Accuracy), but it is essentially free to compute, so the block is retained in case it carries some weight on noisier inputs. The character-n-gram finding also lines up with the smaller 20 Newsgroups improvement seen in Section 5.1: most of the surface variation that character n-grams capture on synthetic data is already represented in the unigram bag on real text.

5.5 Training and Prediction Time

Dataset	Model	Fit time (s)	Predict time (s)
Synthetic	Baseline	0.014	0.004
Synthetic	Solution	0.412	0.063
20 News.	Baseline	0.241	0.054
20 News.	Solution	7.638	1.022

Table 7. Fit and predict times on the held-out 80/20 split, measured with time.perf_counter on a single thread.

Choosing C*=0.1 has a useful side effect: the SVM converges noticeably faster than at the default C=1.0. On 20 Newsgroups the fit time drops from roughly 21 seconds at C=1.0 to under 8 seconds. The NB baseline still runs about 30 times faster than the solution on both datasets, but in absolute terms the solution finishes well under a second on the synthetic corpus and in around eight seconds on the larger 20 Newsgroups split. Both runtimes are well within what would be needed for periodic retraining of a triage tool, so runtime is not a binding constraint here.

5.6 Visualisation

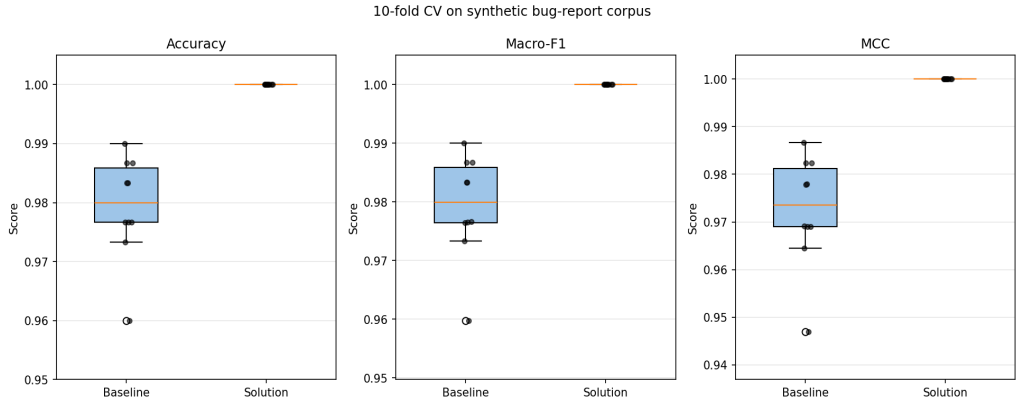

Figure 1. 10-fold CV scores on the synthetic corpus. Baseline (blue) clusters around 0.97-0.99; solution (green) sits at exactly 1.000 on every fold.

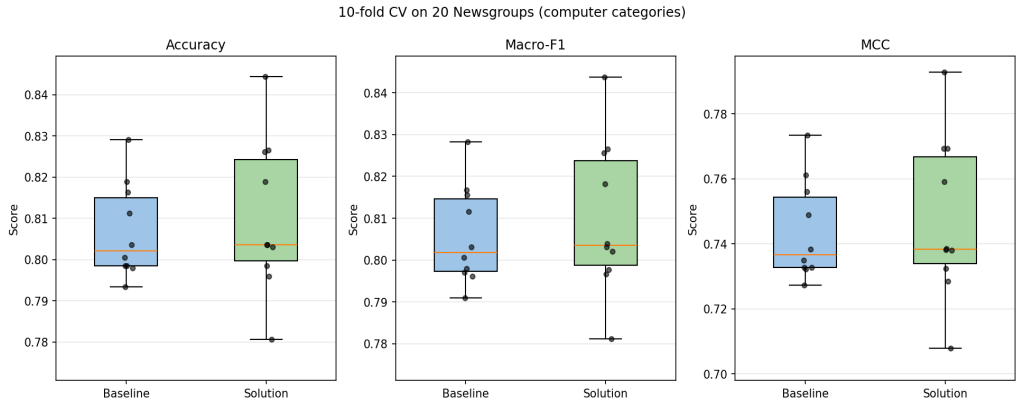

Figure 2. 10-fold CV scores on 20 Newsgroups (computer categories). The two boxes overlap substantially, consistent with the non-significant Wilcoxon result in Table 4.

5.7 Confusion Matrices

Figures 3 and 4 show the full per-class confusion matrices on the 80/20 held-out split, providing cell-by-cell detail beyond the aggregate metrics reported in Table 5.

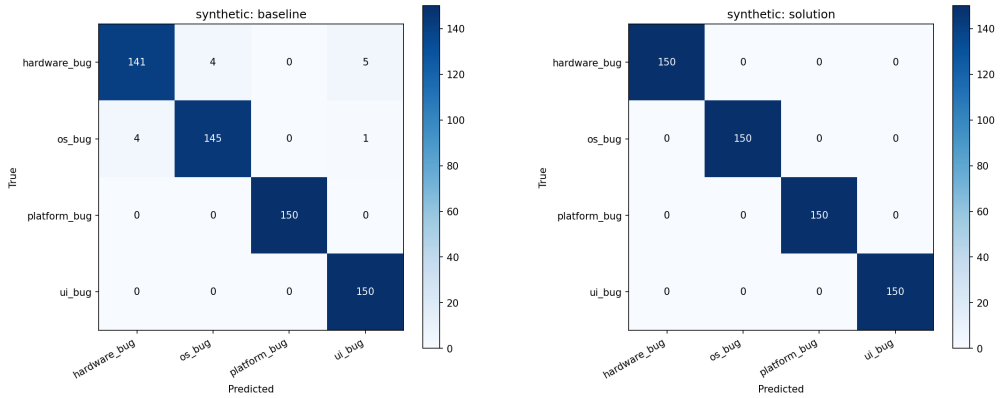

Figure 3. Confusion matrices on the synthetic held-out test (600 reports). Left: Baseline (NB + TF-IDF). Right: Solution (SVC). The solution makes no errors; the baseline confuses platform_bug with os_bug due to overlapping OS and platform vocabulary.

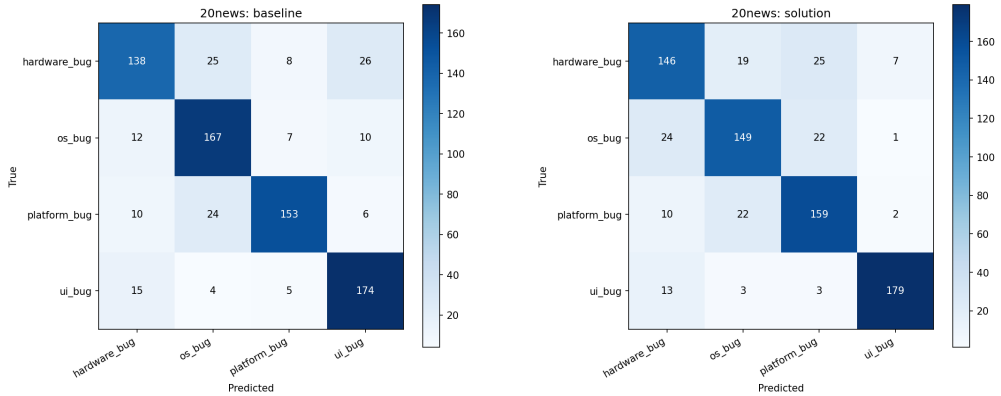

Figure 4. Confusion matrices on the 20 Newsgroups held-out test (784 posts). Left: Baseline (NB + TF-IDF). Right: Solution (SVC). Both classifiers struggle most to separate hardware_bug from platform_bug (adjacent technical vocabularies); the solution redistributes errors, lifting ui_bug recall at a modest cost to platform_bug precision.

6 Reflection

Synthetic vs. 20 News: different stories. The synthetic side of the work shows that, when class boundaries are clean, the enriched feature set together with the tuned SVM beats NB by a margin large enough that the Wilcoxon test bottoms out at p=0.002. On 20 Newsgroups the same advantage compresses down to within fold-to-fold variation once the input is noisy, although the tuned solution is still ahead on all three metrics in the held-out evaluation, which is at minimum directionally consistent. Two factors most likely account for the smaller real-text gap. The first is that real posts already carry many surface variants of the same word, so the sub-word patterns that character n-grams add on synthetic data are largely redundant on real data. The second is length: 20 Newsgroups posts run noticeably longer than a typical bug report and frequently include off-topic content such as signatures and quoted replies that slipped past the standard fetch cleanup, and the small text-statistics block was never designed to filter that content out. The 1.000 score on the synthetic corpus is best read as a best-case ceiling for the feature set, rather than the level of performance to expect on noisy real text. On real data the ceiling lands around 81%, and closing that gap most likely calls for changes to the inputs (such as richer metadata) rather than the classifier.

Limitations. The synthetic baseline already reaches 97.9% Accuracy because the four classes use vocabularies that are different enough for a unigram bag-of-words to almost solve the task on its own; the remaining 2.1 percentage points of headroom are a property of how the synthetic dataset was built rather than an inherent limit of the method. 20 Newsgroups is itself only a proxy for a real bug tracker. The classifier does not see the metadata that a real tracker normally exposes, including reporter history, product, component, version and severity, and DRONE [6] argues that those fields are what actually moves the needle on priority prediction. The character n-gram block adds around 30,000 dimensions, which is fine at these corpus sizes but would call for explicit feature selection, or perhaps truncated SVD, on a tracker containing millions of reports. Finally, the entire pipeline runs at a single random seed of 42. A more thorough study would sweep over several seeds, both to quantify seed-dependent variance in the CV splits and to tighten the resulting confidence intervals.

Threats to validity. *Construct:* the four 20 Newsgroups categories stand in for tracker sub-areas rather than for authentic bug labels, and the synthetic templates necessarily encode a prior view of where the class boundaries should be. *Internal:* the whole pipeline runs at seed 42, so seed-induced variation in the CV splits and in the character-n-gram vocabulary is not quantified, and HPO is performed on a single 75/25 split rather than through nested cross-validation. *External:* the conclusions rest on two corpora and one classifier family, so the feature-contribution findings carry over to other trackers only to the extent that their vocabulary structure is comparable. *Conclusion:* 10 folds give the Wilcoxon test only limited statistical power, which is why effect sizes are reported alongside p-values [11, 13].

Next steps. The most direct next step is to swap 20 Newsgroups for a genuine bug-report dataset, for example along the lines of Antoniol et al. [4] or Mozilla Bugzilla, and to bring in the metadata fields used by DRONE [6]. Richer inputs are the most plausible route to lifting the real-data ceiling above 81% without changing the classifier family. A second avenue is to evaluate a hierarchical attention network along the lines of MHNurf [8], which uses GloVe embeddings and treats title, description, comments and code as parallel streams; that option carries a much heavier compute budget than anything used here. Only after both of those have been exhausted would it be worth fine-tuning a pre-trained transformer [12], which falls outside the CPU-only scope of this tool because it needs GPU resources.

7 Conclusion

Returning to the three research questions raised in Section 1. **RQ1 (synthetic):** yes. The tuned LinearSVC over the enriched TF-IDF features beats the NB plus TF-IDF baseline on all three metrics (1.000 versus 0.979 / 0.979 / 0.973), with the Wilcoxon test at its lower bound (p=0.002) and rank-biserial |r|=1.000 on every metric. The improvement is both statistically significant and large in effect. **RQ2 (20 Newsgroups):** partly. The tuned solution wins on all three metrics on the held-out 80/20 split (+0.0013 Accuracy, +0.0023 Macro-F1, +0.0006 MCC) and on the CV means, but the 10-fold Wilcoxon test is not significant (p=0.6953). The effect size remains small but non-zero (|r|=0.164), so the appropriate interpretation is that the solution is directionally positive but underpowered at this n, rather than equivalent to the baseline. **RQ3 (feature contribution):** character n-grams account for most of the signal in the low-resource synthetic ablation (a drop of 0.020 Accuracy and 0.027 MCC when they are removed); bigrams contribute meaningfully only at full data scale; the text-statistics block is essentially free to compute but adds only marginal gains. On 20 Newsgroups those same additional features add little, because real posts already exhibit the surface variation that the character n-grams are designed to capture. The complete pipeline runs entirely on CPU, finishing in under a second on the synthetic corpus and around eight seconds on 20 Newsgroups, and the source code, raw CSV outputs, figures and one-command replication script are all available in the project repository.

8 Artifact

The source code, raw CSV results and figures live at https://github.com/git-frenzy/ISE-Bug-Classification. The repository root also carries requirements.pdf, manual.pdf and replication.pdf, which together form the supporting documentation for the tool.

References

[1] J. Anvik, L. Hiew, and G. C. Murphy, “Who should fix this bug?” in Proc. ICSE, 2006, pp. 361-370.

[2] T. Chen, “Intelligent Data-Driven Bug Report Analysis,” lecture slides, Intelligent Software Engineering, University of Birmingham, 2025-26.

[3] N. Limsettho, K. Hotta, Y. Higo and S. Kusumoto, “Comparing and combining heuristic approaches for bug report classification,” in Proc. APSEC, 2014, pp. 279-286.

[4] G. Antoniol, K. Ayari, M. Di Penta, F. Khomh and Y.-G. Guéhéneuc, “Is it a bug or an enhancement? A text-based approach to classify change requests,” in CASCON, 2008, pp. 304-318.

[5] T. Joachims, “Text categorization with support vector machines: Learning with many relevant features,” in Proc. ECML, 1998, pp. 137-142.

[6] Y. Tian, D. Lo and C. Sun, “DRONE: Predicting priority of reported bugs by multi-factor analysis,” in Proc. ICSM, 2013, pp. 200-209.

[7] W. B. Cavnar and J. M. Trenkle, “N-gram-based text categorization,” in Proc. SDAIR, 1994, pp. 161-175.

[8] G. Long and T. Chen, “Multifaceted Hierarchical Report Identification for Non-Functional Bugs in Deep Learning Frameworks,” in Proc. APSEC, 2022, pp. 289-298.

[9] F. Wilcoxon, “Individual comparisons by ranking methods,” Biometrics Bulletin, vol. 1, no. 6, pp. 80-83, 1945.

[10] D. Chicco and G. Jurman, “The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation,” BMC Genomics, vol. 21, no. 6, 2020.

[11] A. Arcuri and L. Briand, “A practical guide for using statistical tests to assess randomized algorithms in software engineering,” in Proc. ICSE, 2011, pp. 1-10.

[12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in Proc. NAACL-HLT, 2019, pp. 4171-4186.

[13] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” J. Machine Learning Research, vol. 7, pp. 1-30, 2006.